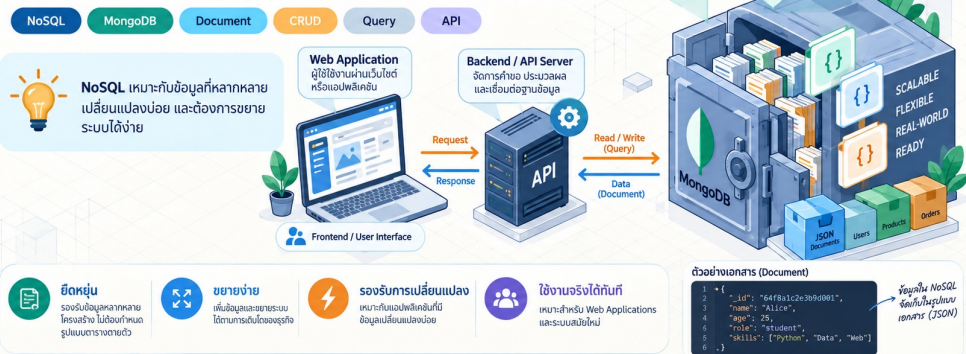


Database Integration using NoSQL

พื้นฐานการเชื่อมต่อฐานข้อมูลแบบยืดหยุ่นสำหรับ Web Applications

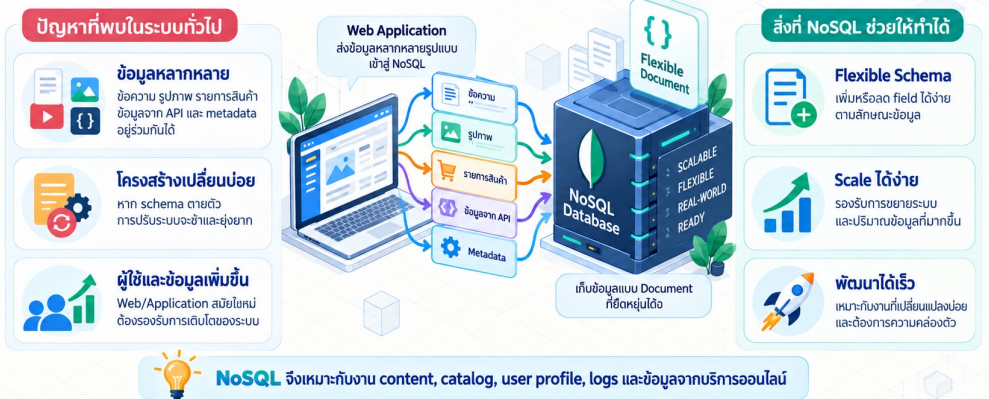


แนวคิดหลัก: เก็บข้อมูลแบบยืดหยุ่น • อ่านง่าย • ขยายง่าย

1

ทำไมต้อง NoSQL?

เมื่อข้อมูลหลากหลายและระบบต้องขยายตัวเร็ว



แนวคิดหลัก: ทำไมต้อง NoSQL • ปัญหา • แนวทางแก้ไข • เหมาะกับงานอะไร

2

SQL vs NoSQL

เลือกเครื่องมือให้เหมาะกับลักษณะข้อมูล



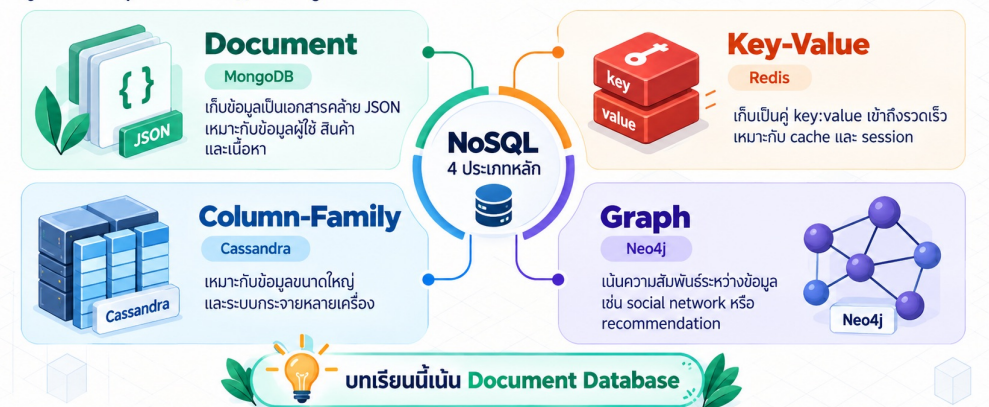
ไม่มีฐานข้อมูลแบบใดที่ดีที่สุดสำหรับทุกงาน — ควรเลือกตามโจทย์ของระบบ

แนวคิดหลัก: เก็บข้อมูลแบบยืดหยุ่น • อ่านง่าย • ขยายง่าย

3

ประเภทของ NoSQL

รู้จัก 4 กลุ่มหลักของฐานข้อมูลแบบไม่สัมพันธ์



แนวคิดหลัก: เก็บข้อมูลแบบยืดหยุ่น • อ่านง่าย • ขยายง่าย

4

Document Database และ JSON/BSON

หัวใจสำคัญของ MongoDB

"ข้อมูลที่จัดระเบียบ
สร้างโอกาสได้มากกว่า"

ตัวอย่างเอกสาร (Document) JSON

```
1 {
2   "name": "Alice",
3   "age": 25,
4   "role": "student",
5   "skills": ["HTML", "CSS", "JavaScript"],
6   "address": { "city": "Bangkok" }
7 }
```

Document ใน MongoDB ข้อมูลที่เกี่ยวข้องกัน เก็บไว้ด้วยกันในเอกสารเดียว

1 Document = 1 รายการข้อมูล
เก็บข้อมูลของสิ่งหนึ่งสิ่งใดไว้รวมกันในเอกสารเดียว

ยืดหยุ่น
แต่ละ document อาจมี field ไม่เหมือนกันทั้งหมด

BSON
MongoDB จัดเก็บภายในเป็น BSON เพื่อให้ประมวลผลได้มีประสิทธิภาพ

เหมาะกับงานจริง
ใช้ได้กับ profile, product, post และข้อมูลจาก API

Document Model (MongoDB)

- เก็บข้อมูลครบในทีเดียว
- ยืดหยุ่น
- รองรับข้อมูลซับซ้อน
- เหมาะกับงานจริง

VS

ตารางแบบเดิม (RDBMS)

- โครงสร้างตายตัว
- เพิ่ม field ต้องแก้ตาราง
- ไม่เหมาะกับข้อมูลซับซ้อน
- ขยายขนาดยาก

Document ตอบโจทย์ยุคใหม่

ตารางแบบเดิมเก็บข้อมูลที่ไม่ตรงร่างขีดยกเว้น

แนวคิดหลัก: เอกสาร (Document) • ยืดหยุ่น • BSON • ใช้งานได้ง่าย

5

โครงสร้างของ MongoDB

Database → Collection → Document → Field

"ข้อมูลที่จัดระเบียบ
สร้างโอกาสได้มากกว่า"

MongoDB
FLEXIBLE
SCALABLE
REAL-WORLD
READY

Database
กลุ่มข้อมูลขนาดใหญ่ของระบบ

Collection
กลุ่ม document ประเภทเดียวกัน

Document
ข้อมูล 1 ชิ้น เช่น ผู้ใช้ 1 คน

Field
คุณลักษณะภายใน document

ตัวอย่าง florastore

ตัวอย่าง users

ตัวอย่าง
{ name: 'Alice', age: 25, role: 'admin' }

ตัวอย่าง
name, age, role

เปรียบเทียบให้เห็นภาพ

Database = กล่องใหญ่
เก็บข้อมูลทั้งหมดของระบบ

Collection = แฟ้มข้อมูล
จัดกลุ่มเอกสารประเภทเดียวกัน

Document = เอกสาร 1 ใบ
ข้อมูล 1 ชิ้น เช่น ผู้ใช้ 1 คน

name, age, role

Field = รายละเอียดในเอกสาร
คุณลักษณะต่าง ๆ เช่น ชื่อ อายุ บทบาท

แนวคิดหลัก: เก็บข้อมูลแบบยืดหยุ่น • ใช้งานง่าย • ขยายง่าย

6

CRUD ใน MongoDB

วงจรพื้นฐานของการจัดการข้อมูล

"ข้อมูลที่จัดระเบียบ
เริ่มจากการจัดการที่ถูกต้อง"

01 Create
insertOne(), insertMany()
เพิ่มข้อมูลใหม่เข้าสู่ collection

02 Read
find(), findOne()
อ่านและค้นหาข้อมูลที่ต้องการ

03 Update
updateOne(), updateMany()
แก้ไขข้อมูลเดิมให้เป็นปัจจุบัน

04 Delete
deleteOne(), deleteMany()
ลบข้อมูลที่ไม่ต้องการ

แนวคิดเหมือน SQL
แต่รูปแบบคำสั่งถูกออกแบบมาสำหรับเอกสาร (document)

ใช้งานง่าย
โครงสร้างที่เรียบง่าย เรียนรู้ได้รวดเร็ว

ยืดหยุ่น
รองรับข้อมูลหลากหลายรูปแบบ (Document)

พร้อมสำหรับอนาคต
เหมาะกับการปรับขนาดในสเกลใหญ่ ที่ต้องการความคล่องตัว

ตอบโจทย์ทุกขนาด
ตั้งแต่โปรเจกต์เล็ก ไปจนถึงระดับองค์กร

MongoDB
SCALABLE
FLEXIBLE
REAL-WORLD
READY

แนวคิดหลัก: การจัดการข้อมูลด้วย CRUD • สร้าง • อ่าน • แก้ไข • ลบ

7

เพิ่มและอ่านข้อมูล

Create + Read ใน MongoDB

"ข้อมูลที่จัดระเบียบ
สร้างโอกาสได้มากกว่า"

Create
เพิ่มข้อมูล (Create) ลงในคอลเลกชัน

1. เพิ่มข้อมูล 1 รายการ
1 > db.users.insertOne({ name: 'Alice', age: 25, city: 'Bangkok' })

2. เพิ่มข้อมูลหลายรายการ
1 > db.users.insertMany([{ name: 'Bob', age: 30, city: 'Bangkok' }, { name: 'Charlie', age: 35, city: 'Bangkok' }])

ใช้สำหรับเพิ่ม document หนึ่งรายการหรือหลายรายการ

เพิ่มข้อมูลเข้าไว้ในคอลเลกชัน users

Read
อ่านข้อมูล (Read) จากคอลเลกชัน

1. อ่านข้อมูลทั้งหมด
1 > db.users.find()

2. อ่านข้อมูลแบบมีเงื่อนไข (filter)
1 > db.users.find({ age: { \$gte: 25 } })

3. อ่านข้อมูลแบบเลือกเฉพาะ field ที่ต้องการ (projection)
1 > db.users.find({ city: 'Bangkok' }, { name: 1, age: 1, _id: 0 })

filter = เงื่อนไขค้นหา

projection = เลือก field ที่ต้องการแสดง

Query = Read Document(s)

1 > db.users.find({ city: 'Bangkok' }, { name: 1, age: 1, _id: 0 })

✓ ได้ข้อมูลตามเงื่อนไข จากคอลเลกชัน users

แนวคิดหลัก: เพิ่มข้อมูลและอ่านข้อมูล • Create • Read • MongoDB

8

แก้ไขและลบข้อมูลอย่างปลอดภัย

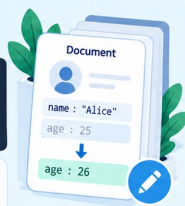
Update + Delete พร้อมข้อควรระวัง

NoSQL MongoDB Document CRUD Query API

Update

```
1 db.users.updateOne(
2   { name: 'Alice' },
3   { $set: { age: 26 } }
4 )
```

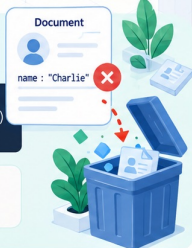
ค้นหา document ที่ต้องการ แล้วแก้ไขเฉพาะ field ที่ระบุ



Delete

```
1 db.users.deleteOne({ name: 'Charlie' })
```

ลบ document ที่ตรงตามเงื่อนไข



“ข้อมูลที่ดีและปลอดภัย ช่วยให้ระบบของคุณมั่นคงกว่า”



ข้อควรระวัง

การแก้ไขและลบข้อมูลเป็นการเปลี่ยนแปลงทาง การตรวจสอบให้รอบคอบก่อนดำเนินการ



กำหนดเงื่อนไข ให้ชัดเจน



ลองใช้ find() เพื่อตรวจสอบก่อน



คำสั่งที่ไม่มี filter อาจกระทบข้อมูล จำนวนมาก



สำรองข้อมูลก่อน แก้ไขข้อมูลสำคัญ

แนวคิดหลัก: แก้ไขและลบข้อมูล • ข้อควรระวัง • ใช้งานอย่างปลอดภัย

9

Web Application + NoSQL Integration

ภาพรวมการทำงานตั้งแต่ผู้ใช้งานถึงฐานข้อมูล



ตัวอย่างการใช้งาน (Use Cases)

User Profile จัดเก็บข้อมูลผู้ใช้ที่หลากหลาย

Product Catalog จัดเก็บข้อมูลสินค้าที่มีหลายรูปแบบ

Comments จัดเก็บความคิดเห็นจากผู้ใช้งาน

Logs จัดเก็บข้อมูลบันทึกการใช้งานระบบ

NoSQL ช่วยพัฒนา Web Applications ได้อย่างยืดหยุ่น ขยายระบบได้ง่าย และรองรับข้อมูลจริงที่เปลี่ยนแปลงตลอดเวลา

สรุปจุดเด่น

Flexible รองรับข้อมูลหลากหลายรูปแบบ

Scalable ขยายระบบได้ง่ายตามการเติบโต

Developer Friendly เหมาะสำหรับการพัฒนา สร้างแอปได้เร็ว

แนวคิดหลัก: เชื่อมข้อมูลแบบยืดหยุ่น • ใช้งานง่าย • ขยายง่าย

10